Proseminar: Primzahltest und Berechenbarkeit

Jana Chung

15. April 2026

Inhaltsverzeichnis

1. Was ist ein Algorithmus
2. Berechnungsproblem
3. Pseudocode
4. Primzahltest
5. Sieb des Eratosthenes
6. Halteproblem

1 Was ist ein Algorithmus

Ein Algorithmus ist eine endliche, wohldefinierte Folge von Anweisungen, die für jede zulässige Eingabe nach endlich vielen Schritten terminiert und eine Ausgabe liefert

- Welche Eingaben akzeptiert er?
- Welche Rechenschritte werden in welcher Reihenfolge durchgeführt?
- Wann stoppt der Algorithmus? Was wird dann ausgegeben?

2 Berechnungsprobleme

Durch eine Funktion $f\colon A \to B$ können wir ausdrücken dass ein Computerprogramm bei Eingabe eines Inputs $a \in A$ das Ergebnis $f(a) \in B$ ausgibt.

Definition 1

Sei A eine nichtleere endliche Menge. Für $k \in \mathbb{N} \cup \{0\}$ bezeichnen wir mit A^k die Menge der Funktionen $f : \{1, \ldots, k\} \to A$. Ein Element f $\in A^k$ heißt **Wort / Zeichenkette** der **Länge** k über dem **Alphabet** A. Wir schreiben es als Folge $f(1)...f(k)$. Das einzige Element $\emptyset$ von A^0 nennen wir das **leere Wort** (mit Länge 0). Wir setzen $A^* := \bigcup_{k \in \mathbb{N} \cup \{0\}} A^k$. Eine **Sprache** über dem Alphabet A ist eine Teilmenge von A^*.

2 Berechnungsprobleme

Definition 2

Ein **Berechnungsproblem** ist eine Relation $P \subseteq D \times E$, wobei es zu jedem d $\in$ D mindestens ein e $\in$ E gibt mit $(d, e) \in$ P. Wenn $(d, e) \in$ P, dann ist e eine **korrekte Ausgabe** für das Problem P mit Eingabe d. Die Elemente von D heißen **Instanzen** des Problems. P heißt **eindeutig**, wenn P eine Funktion ist. Sind D, E Sprachen über einem endlichen Alphabet A, so sprechen wir von einem **diskreten Berechnungsproblem**. Sind D und E Teilmengen von $\mathbb{R}^m$ bzw. $\mathbb{R}^n$ für $m, n \in \mathbb{N}$, so sprechen wir von einem **numerischen Berechnungsproblem**. Ein eindeutiges Berechnungsproblem $P\colon D \to E$ mit $|E| = 2$ heißt **Entscheidungsproblem**.

3 Pseudocode

Algorithmen sollen Berechnungsprobleme lösen, heutzutage erfolgt die Ausführung mit Computern.

- Algorithmus entwerfen
- wird in Computerprogramm (z.B. in C++) implementiert
- wird zu Maschinencode kompiliert
- Ausführung

Pseudocode: macht einen Algorithmus konkret in einer noch für Menschen verständlichen Sprache, und gleichzeitig leicht in eine Programmiersprache umsetzbar. Gut um Idee des Algorithmus darzustellen

4 Primzahltest - Was heißt prim?

Der Algorithmus soll herausfinden, ob eine gegebene natürliche Zahl **prim ist oder nicht**.

Entscheidungsproblem: $\{(n, e) \in \mathbb{N} \times \{0, 1\} | \, e = 1 \iff \text{ n prim } \}$.

Definition 3

Eine natürliche Zahl n heißt prim, wenn $n \geq 2$ ist und es keine natürliche Zahl a und b gibt mit $a > 1, b > 1$ und $n = a \cdot b$

4 Primzahltest - Code und Definition

- **If-Anweisung:** Führt Anweisungen abhängig vom Wahrheitswert einer Bedingung aus. Ist die Bedingung wahr, wird der `then`-Teil ausgeführt, andernfalls der `else`-Teil (optional)
- **For-Anweisung:** Eine Variable durchläuft sukzessive Werte, und für jeden Wert wird ein Anweisungsblock ausgeführt.

Definition 4 (Gaußklammern)

Für $x \in \mathbb{R}$ definieren wir:
$\lfloor x \rfloor := \max\{k \in \mathbb{Z} | k \leq x\}, \quad \lceil x \rceil := \min\{k \in \mathbb{Z} | k \geq x\}.$

Definition 5 (Modulo-Operation)

Für $a, b \in \mathbb{N}$ definieren wir: $a \bmod b := a - b \cdot \lfloor \frac{a}{b} \rfloor$. Es gilt
$b | a \iff a \bmod b = 0.$

4 Primzahltest - Algorithmus

Nun erfolgt der Algorithmus, ob eine natürliche Zahl prim ist oder nicht in Pseudocode

---

Require: $n \in \mathbb{N}$
Ensure: Antwort auf die Frage, ob n prim ist

```
if n = 1 then
    antwort ← "nein"
else
    antwort ← "ja"
    for i ← 2 to ⌊√n⌋ do
        if i | n then
            antwort ← "nein"
return antwort
```

Überblick

Eratosthenes
- Algorithmus
- Beweis

Halteproblem
- Kodierung von Wörtern
- Abzählbarkeit
- Halteproblem
- Widerspruchsbeweis

5 Sieb des Eratosthenes – Überblick

- Algorithmus
- Beweis

Überblick

Eratosthenes
- Algorithmus
- Beweis

Halteproblem
- Kodierung von Wörtern
- Abzählbarkeit
- Halteproblem
- Widerspruchsbeweis

5 Sieb des Eratosthenes - Algorithmus

Nach Eingabe einer natürlichen Zahl n besteht die Aufgabe darin, alle Primzahlen p mit p $\leq$ n zu berechnen. Der Algorithmus lautet wie folgt:

Require: $n \in \mathbb{N}$
Ensure: alle Primzahlen $\leq n$

```
for i ← 2 to n do
    p[i] ← "ja"
for i ← 2 to n do
    if p[i] = "ja" then
        for j ← i to ⌊n/i⌋ do
            p[i · j] ← "nein"
return alle i mit p[i] = "ja"
```

---

5 Sieb des Eratosthenes I

Satz 1

Obiger Algorithmus ist korrekt.

Wir zeigen die Korrektheit des Algorithmus. Dazu beweisen wir für alle $k \in \{2, \ldots, n\}$:

k **wird genau dann ausgegeben, wenn** k **prim ist.**

Dies beweisen wir indem wir folgende 2 Aussagen beweisen

- Ist k prim, so wird k ausgegeben (1)
- Ist k nicht prim, so wird k nicht ausgegeben (2)

Überblick

Eratosthenes
- Algorithmus
- Beweis

Halteproblem
- Kodierung von Wörtern
- Abzählbarkeit
- Halteproblem
- Widerspruchsbeweis

5 Sieb des Eratosthenes

Beweis 1.

(1) Ist k prim, so wird k ausgegeben.

Sei $k \in \{2, \ldots, n\}$ eine Primzahl. Im Algorithmus wird eine Zahl k genau dann nicht ausgegeben, wenn irgendwann $p[k] \leftarrow$ "nein" gesetzt wird. Wir zeigen, dass dies für Primzahlen nicht passiert. Die Zuweisung $p[k] \leftarrow$ "nein" erfolgt nur dann, wenn es Zahlen $i, j \in \mathbb{N}$ mit $i \geq 2$, $j \geq i$ gibt, sodass

$$k = i \cdot j.$$

Das bedeutet aber, dass k als Produkt zweier Zahlen größer oder gleich 2 dargestellt werden kann. Dies widerspricht der Annahme, dass k prim ist. Also wird für eine Primzahl k niemals $p[k] =$ "nein" gesetzt. Somit gilt während des gesamten Algorithmus:

$$p[k] = \text{“ja”}.$$

Folglich wird k ausgegeben.

5 Sieb des Eratosthenes

(2) Ist k nicht prim, so wird k nicht ausgegeben.
Sei nun $k \in \{2, \ldots, n\}$ keine Primzahl. Dann ist k zusammengesetzt, also existieren Zahlen $i, j \in \mathbb{N}$ mit

$$k = i \cdot j \quad \text{und} \quad 2 \leq i \leq j.$$

Wir wählen i als den kleinsten Teiler von k, der mindestens 2 ist. Dann gilt insbesondere:

$$2 \leq i \leq j = \frac{k}{i}.$$

Da i der kleinste Teiler von k ist, kann i selbst keine nichttrivialen Teiler besitzen. Also ist i eine Primzahl.
Nach Teil (1) wissen wir, dass für Primzahlen stets $p[i] =$ "ja" gilt, wenn der Algorithmus die Zahl i betrachtet.

5 Sieb des Eratosthenes

In diesem Schritt führt der Algorithmus die innere Schleife aus und setzt für alle geeigneten j:

$$p[i \cdot j] \leftarrow \text{“nein”}.$$

Da $k = i \cdot j$, wird insbesondere auch

$$p[k] \leftarrow \text{“nein”}$$

gesetzt. Ab diesem Zeitpunkt bleibt $p[k] = \text{“nein”}$, da dieser Wert im Algorithmus nicht mehr geändert wird.
Wenn später im äußeren Durchlauf der Wert $i = k$ erreicht wird, gilt daher:

$$p[k] = \text{“nein”},$$

und somit wird k nicht ausgegeben.

Damit ist die Korrektheit des Algorithmus gezeigt. $\square$

Überblick

Eratosthenes
- Algorithmus
- Beweis

Halteproblem
- Kodierung von Wörtern
- Abzählbarkeit
- Halteproblem
- Widerspruchsbeweis

6 Halteproblem – Überblick

- Kodierung von Wörtern (Lemma)
- Abzählbarkeit von Programmen
- Existenz nicht berechenbarer Funktionen
- Definition der Haltefunktion
- Widerspruchsbeweis

Überblick

Eratosthenes
- Algorithmus
- Beweis

Halteproblem
- Kodierung von Wörtern
- Abzählbarkeit
- Halteproblem
- Widerspruchsbeweis

6 Halteproblem

Zunächst benötigen wir einige Vorarbeit.

Lemma 2

Seien $k, l \in \mathbb{N}$. Definiere die Abbildung

$$f^l : \{0, \ldots, k-1\}^l \to \{0, \ldots, k^l - 1\}, \quad f^l(w) := \sum_{i=1}^{l} a_i k^{l-i}.$$

für alle $w = a_1, \ldots, a_l \in \{0, ..., k-1\}^l$. Dann ist die Abbildung f^l wohldefiniert und bijektiv.

6 Halteproblem

Beweis 2.

1. Wohldefiniertheit:

Sei $(a_1, \ldots, a_l) \in \{0, \ldots, k-1\}^l$. Dann gilt für alle i:

$$0 \leq a_i \leq k-1.$$

Damit folgt

$$0 \leq a_i k^{l-i} \leq (k-1)k^{l-i}.$$

Durch Aufsummieren erhalten wir

$$0 \leq f_l(a_1, \ldots, a_l) = \sum_{i=1}^{l} a_i k^{l-i} \leq \sum_{i=1}^{l} (k-1)k^{l-i}.$$

Die rechte Summe ist eine geometrische Reihe:

6 Halteproblem

Wir formen die rechte Summe um

$$\sum_{i=1}^{l}(k-1)k^{l-i} = (k-1)\sum_{i=1}^{l} k^{l-i}$$

Wir substituieren den Laufindex durch $j := l - i$ und ersetzen die Indexgrenzen. Für $i = 1$ erhalten wir die neue indexgrenze $j = l - 1$ und für $i = l$ erhalten wir die neue Indexgrenze $j = 0$.

$$= (k-1)\sum_{j=0}^{l-1} k^j = (k-1)\cdot\frac{k^l - 1}{k-1} = k^l - 1.$$

Also gilt insgesamt

$$0 \leq f_l(a_1, \ldots, a_l) \leq k^l - 1,$$

und damit ist f_l wohldefiniert.

6 Halteproblem

2. Injektivität:

Seien $(a_1, \ldots, a_l), (b_1, \ldots, b_l) \in \{0, \ldots, k-1\}^l$ mit

$$(a_1, \ldots, a_l) \neq (b_1, \ldots, b_l)$$

beliebig aber fest gewählt. Dann gibt es einen kleinsten Index $j \in \{1, \ldots, l\}$ mit

$$a_j \neq b_j.$$

Ohne Einschränkung sei $a_j > b_j$.

$$f^l(a_1, \ldots, a_l) - f^l(b_1, \ldots, b_l) = \sum_{i=1}^{l} (a_i - b_i) k^{l-i}.$$

Für $i < j$ gilt $a_i = b_i$, also verschwinden diese Terme:

$$= (a_j - b_j) k^{l-j} + \sum_{i=j+1}^{l} (a_i - b_i) k^{l-i}.$$

6 Halteproblem

Wir betrachten die Differenz:

Nun schätzen wir die Summe für $i > j$ nach unten ab. Da $a_i, b_i \in \{0, \ldots, k-1\}$ gilt

$$a_i - b_i \geq -(k-1).$$

Also

$$\sum_{i=j+1}^{l} (a_i - b_i) k^{l-i} \geq -\sum_{i=j+1}^{l} (k-1) k^{l-i}.$$

Wie oben ergibt die geometrische Summe:

$$\sum_{i=j+1}^{l} (k-1) k^{l-i} = k^{l-j} - 1.$$

Damit folgt

$$\sum_{i=j+1}^{l} (a_i - b_i) k^{l-i} \geq -(k^{l-j} - 1).$$

6 Halteproblem

Zusammen ergibt sich:

$$f^l(a_1, \ldots, a_l) - f^l(b_1, \ldots, b_l) \geq (a_j - b_j)k^{l-j} - (k^{l-j} - 1).$$

Da $a_j > b_j$, gilt $a_j - b_j \geq 1$, also:

$$\geq k^{l-j} - (k^{l-j} - 1) = 1.$$

Insbesondere ist

$$f^l(a_1, \ldots, a_l) > f^l(b_1, \ldots, b_l).$$

Also ist f^l injektiv.

3. Surjektivität:
Die Mengen haben dieselbe Mächtigkeit:

$$|\{0, \ldots, k-1\}^l| = k^l = |\{0, \ldots, k^l - 1\}|.$$

Die linke Menge besteht aus allen Tupeln der Länge l. Für jede Position gibt es k verschiedene Möglichkeiten. Die Anzahl ergibt sich also als Produkt $k \cdot k \cdot k \ldots k = k^l$. Die rechte Menge besteht einfach aus allen Zahlen von 0 bis $k^l - 1$, weist also auch die Anzahl k^l auf (am Ende zieht man zwar 1 ab, fängt aber mit 0 an).

6 Halteproblem

Da f^I eine injektive Abbildung zwischen zwei endlichen Mengen gleicher Größe ist, ist sie automatisch auch surjektiv.

f^I ist somit bijektiv. $\square$

Überblick

Eratosthenes
- Algorithmus
- Beweis

Halteproblem
- Kodierung von Wörtern
- Abzählbarkeit
- Halteproblem
- Widerspruchsbeweis

6 Halteproblem

Satz 3

Sei A eine endliche nichtleere Menge. Dann ist die Menge A^* abzählbar.

Der Beweis wird in Folge nur skizziert:

6 Halteproblem

Beweis 3. Die Idee des Beweises besteht darin, die Wörter aus A^* systematisch zu nummerieren.
Zunächst werden alle Wörter nach ihrer Länge geordnet. Für jedes $l \in \mathbb{N}$ gibt es genau $|A|^l$ Wörter der Länge l. Da A endlich ist, ist somit auch jede Menge A^l endlich.
Anschließend werden die Wörter fester Länge l eindeutig als Zahlen im $|A|$-adischen Zahlensystem kodiert. Dazu wird jedem Symbol aus A eine Ziffer aus $\{0, \ldots, |A| - 1\}$ zugeordnet.
Durch diese Kodierung erhält man eine injektive Abbildung von A^* nach $\mathbb{N}$, da Wörter unterschiedlicher Länge in verschiedene Bereiche fallen und die Darstellung innerhalb fester Länge eindeutig ist.
Damit ist A^* abzählbar. $\square$

6 Halteproblem

Korollar 4

Die Menge aller C++-Programme ist abzählbar.

Beweis 4. C++-Programme sind Wörter über einem endlichen Alphabet. Die zu zeigende Aussage folgt also aus Satz 4. □

6 Halteproblem

Satz 5

Es gibt Funktionen $f : \mathbb{N} \to \{0, 1\}$, die von keinem C++-Programm berechnet werden.

Beweis 5. Sei $\mathcal{P}$ die nach Korrolar 4 abzählbare Menge der C++-Programme, die eine Funktion $f : \mathbb{N} \to \{0, 1\}$ berechnen. Sei $g : \mathcal{P} \to \{0, 1\}$ eine injektive Funktion. Für $P \in \mathcal{P}$ sei f^P die von P berechnete Funktion.
Betrachte eine Funktion $f : \mathbb{N} \to \{0, 1\}$ mit $f(g(P)) := 1 - f^P(g(P))$ für alle $P \in \mathcal{P}$. Eine solche existiert, da g injektiv ist. Offenbar ist $f \neq f^P$ für alle $P \in \mathcal{P}$, und somit wird f von keinem C++ Programm berechnet. $\square$

Überblick

Eratosthenes
- Algorithmus
- Beweis

Halteproblem
- Kodierung von Wörtern
- Abzählbarkeit
- Halteproblem
- Widerspruchsbeweis

6 Halteproblem

Unter den nicht berechenbaren Funktionen geht es nun um das berühmteste Beispiel, dem **Halteproblem**: Es gibt kein Programm, dass für alle Programme und Eingaben entscheiden kann, ob das Programm terminiert.

Wir betrachten die natürlichen Zahlen $\mathbb{N}$ und die Menge aller Programme Q. Da Programme endliche Texte darstellen, können wir alle Programme durchnummerieren. Es gibt also eine surjektive Funktion $g : \mathbb{N} \to Q$ - jede Zahl kodiert ein Programm. Genauso geht die Kodierung auch in die andere Richtung, jedes Programm kodiert eine Zahl: $f : Q \to \mathbb{N}$. Jedem Programm wird eine Zahl zugeordnet.

6 Halteproblem

Wir definieren die Funktion wie folgt:

$$h : \mathbb{N} \times \mathbb{N} \rightarrow \{1, 0\} \tag{1}$$

$$h(x, y) = \begin{cases} 1, & \text{wenn Programm g(x) bei Eingabe y terminiert} \\ 0, & \text{sonst} \end{cases} \tag{2}$$

x: Nummer eines Programms; y: Eingabe; h(x,y) sagt ob Programm terminiert oder nicht.

Diese Funktion heißt **Haltefunktion** und ihr zugehöriges Entscheidungsproblem heißt **Halteproblem**.

Überblick

Eratosthenes
- Algorithmus
- Beweis

Halteproblem
- Kodierung von Wörtern
- Abzählbarkeit
- Halteproblem
- Widerspruchsbeweis

6 Halteproblem

Satz 6

Die Haltefunktion wird von keinem C++ Programm berechnet. Das heißt es existiert kein Programm das h berechnet.

Beweis 6. Die Beweisidee ist ein Widerspruchsbeweis. Angenommen, es existiert ein Programm P, das $h(x, y)$ korrekt berechnet. Wir konstruieren ein neues Programm Q, das wie folgt funktioniert:
Bei Eingabe $x \in \mathbb{N}$:

1. Berechne h(x,x)
2. Falls $h(x, x) = 0 \rightarrow$ terminiert
3. falls $h(x, x) = 1 \rightarrow$ terminiert nicht

Schlüsselidee

Es passiert genau das Gegenteil von dem, was h vorhersagt.

6 Halteproblem

Da Q ein Programm ist und jedes Programm eine Nummer hat, gibt es ein q mit $g(q) = Q$.
Nun betrachten wir $h(q, q)$.

- **Fall 1:** Betrachte $h(q, q) = 1$. $Q(q)$ terminiert, aber nach Definition von Q terminiert es nicht.
- **Fall 2:** Betrachte $h(q, q) = 0$. $Q(q)$ terminiert es nicht, aber nach Definition von Q müsste es terminieren.

In beiden Fällen entsteht also ein Widerspruch. Also war die Annahme falsch. Die Haltefunktion ist somit nicht berechenbar.

Man sagt dazu: das Halteproblem ist **nicht entscheidbar**. $\square$

6 Halteproblem

Ein gutes Beispiel für ein Programm, bei dem nicht leicht zu sehen ist, ob es hält, ist die **Collatz-Folge**. Der Algorithmus funktioniert wie folgt: Nach Eingabe von $n \in \mathbb{N}$ wird berechnet, ob die Collatz-Folge von n den Wert 1 erreicht. Dabei geht das Programm wie folgt vor. Wenn n eine gerade Zahl ist, so wird n halbiert. Ansonsten wird es verdreifacht und um 1 erhöht. Dies wiederholt sich bis die Collatz-Folge den Wert 1 erreicht. Bis heute konnte nicht erfasst werden, ob dieser Algorithmus immer, also für jedes n, hält.

Literatur

Hauptquelle:

- Stefan Hougardy und Jens Vygen:
  Algorithmische Mathematik
  Springer, 2010

Bemerkung:

- Grundlage für alle Definitionen und Beweise
- Einige Beweisschritte wurden im Vortrag ausführlicher dargestellt